Architecture Proposal: Madagascar Freshwater Fish Conservation Platform

Date: 2026-04-09 **Status:** Draft -- Awaiting Human Review

System Overview

The Madagascar Freshwater Fish Conservation Platform is an open-source web application that serves as the centralized data infrastructure for Madagascar's approximately 79 endemic freshwater fish species -- one of the world's most threatened freshwater faunas, with 63% of species facing extinction. The platform integrates four capabilities that no existing system combines: (a) public-facing species profiles drawing from IUCN, FishBase, and field data; (b) restricted coordination tools for tracking ex-situ populations across zoos, aquariums, and private breeders; (c) field program monitoring that links in-situ surveys to ex-situ management; and (d) cross-sector networking between professional institutions and hobbyist communities. It complements rather than replaces existing systems (ZIMS, IUCN Red List, FishBase, GBIF), filling the documented gap in which data fragmentation across a dozen disconnected systems is explicitly identified in peer-reviewed literature as a barrier to preventing extinctions.

The platform serves a small but internationally distributed community: Malagasy researchers, European and North American zoo staff, TAG coordinators, hobbyist breeders (CARES, Citizen Conservation), SHOAL and IUCN FFSG leadership, and in-country NGO partners. Public-facing species profiles serve a broader audience including educators, policymakers, journalists, and conservation funders.

Technical Stack

Layer	Technology	Rationale
Language	Python 3.12+	Dominant language in biodiversity informatics; strong geospatial libraries (GDAL, Shapely, GeoPandas); large contributor pool in conservation tech; specified in CLAUDE.md
Web Framework	Django 5.x	Mature ORM with excellent PostgreSQL/PostGIS support; built-in admin interface accelerates early data entry; robust auth framework supports the five-tier access model; battle-tested in scientific data platforms (GBIF tools, ALA)
Database	PostgreSQL 16 + PostGIS 3.4	Industry standard for biodiversity data; native spatial queries for species distribution mapping and coordinate generalization; JSONB for flexible Darwin Core extensions; recommended by the gap analysis and aligned with BiOS, OpenBioMaps, and other modern biodiversity platforms
API	Django REST Framework (DRF)	Decoupled RESTful API enabling independent frontend development; serializer-level field filtering supports tiered data visibility; built-in OpenAPI/Swagger documentation; Darwin Core Archive export endpoints

Frontend	Next.js 14 (React, TypeScript)	Server-side rendering for SEO on public species profiles; React component model for complex coordination dashboards; TypeScript for type safety on Darwin Core field contracts; static generation for species pages reduces server load
Search	PostgreSQL full-text search (initial); Meilisearch (future)	Full-text search on species names (scientific + common, multilingual: English, French, Malagasy) is a core requirement; PostgreSQL FTS is sufficient at ~200 species scale; Meilisearch adds typo tolerance and faceted filtering if needed later
GIS/Mapping	Leaflet + PostGIS	Leaflet is lightweight, open-source, and well-suited for species distribution maps; PostGIS handles server-side coordinate generalization for sensitive locations; no dependency on proprietary mapping services
Task Queue	Celery + Redis	Background processing for Darwin Core Archive generation, GBIF publishing, external API synchronization (IUCN, FishBase), and data import/export jobs
File Storage	S3-compatible object storage (MinIO for dev, AWS S3 or Backblaze B2 for production)	Species images, field survey photos, documents, Darwin Core Archive files
Caching	Redis	Session storage, API response caching for external integrations (IUCN/FishBase responses cached to avoid rate limits), page fragment caching for species profiles

Stack Rationale Summary

The Django/PostgreSQL-PostGIS/DRF combination is chosen over alternatives for three reasons specific to this project:

1. **Biodiversity ecosystem alignment.** The gap analysis identifies Darwin Core as the mandatory data standard and PostgreSQL-PostGIS as the recommended backend. Django's ORM maps cleanly to Darwin Core's relational structure, and PostGIS is the only mature open-source spatial database that supports the coordinate generalization protocols required by GBIF sensitive species best practices.

2. **Admin-first development.** The community's immediate need (documented in the January 2026 workshop notes) is a centralized database for species/research data. Django's built-in admin interface allows conservation coordinators to begin entering and managing data before the full frontend is complete -- critical for a project where the user community is small, technically mixed, and needs to see value quickly.

3. **Contributor accessibility.** Python/Django has the broadest contributor base in conservation technology. Given that this is an open-source project reliant on community contributions, framework familiarity matters more than marginal performance gains from alternatives.

Component Architecture

The system follows a layered architecture with a clear API boundary between backend and frontend, enabling independent deployment and allowing the API to serve future mobile clients or third-party integrations.

Components

```
+-------------------------------------------------------------------+
|                        PUBLIC INTERNET                            |
+-------------------------------------------------------------------+
         |                    |                      |
         v                    v                      v
+----------------+  +------------------+  +-------------------+
|   Next.js      |  |  DRF REST API    |  |  Django Admin     |
|   Frontend     |  |  (JSON + DwC)    |  |  (Data Entry)     |
|   (SSR + SPA)  |  |                  |  |                   |
+----------------+  +------------------+  +-------------------+
         |                    |                      |
         +--------------------+----------------------+
                              |
                    +---------v----------+
                    |   Django App Core  |
                    |                    |
                    |  +-- species ------+--  Species profiles, taxonomy,
                    |  |                 |    IUCN status, FishBase data
                    |  +-- populations --+--  Ex-situ populations, institutions,
                    |  |                 |    breeding records, transfers
                    |  +-- fieldwork ----+--  Field programs, surveys,
                    |  |                 |    occurrence records, eDNA data
                    |  +-- coordination -+--  Breeding recommendations,
                    |  |                 |    prioritization scores, action items
                    |  +-- accounts -----+--  Users, institutions, roles,
                    |  |                 |    access tiers, audit log
                    |  +-- integration --+--  External API sync, DwC export,
                    |                    |    GBIF publishing
                    +--------------------+
                              |
              +---------------+----------------+
              |               |                |
     +--------v---+    +-----v------+   +-----v------+
     | PostgreSQL  |    |   Redis    |   |   S3/MinIO |
     | + PostGIS   |    |  (cache +  |   |  (files)   |
     |             |    |   queue)   |   |            |
     +-------------+    +------------+   +------------+
              |
     +--------v-----------+
     | Celery Workers      |
     | - IUCN/FishBase sync|
     | - DwC Archive gen   |
     | - GBIF publishing   |
```

```
    | - Report generation |
    +---------------------+
```

Django Applications

Application	Responsibility	Key Models
species	Taxonomic data, species profiles, conservation status, distribution, ecology, images	Species, Taxon, ConservationAssessment, Distribution, Habitat, CommonName
populations	Ex-situ population tracking across institutions and private breeders	ExSituPopulation, Institution, HoldingRecord, BreedingEvent, Transfer, Census
fieldwork	In-situ conservation programs, field surveys, occurrence records	FieldProgram, Survey, OccurrenceRecord, SiteMonitoring, eDNASample
coordination	Conservation planning, breeding recommendations, prioritization	ConservationAction, BreedingRecommendation, PrioritizationScore, ActionPlan
accounts	User management, institutional affiliations, access tier enforcement	User, InstitutionMembership, AccessTier, AuditLog
integration	External API clients, Darwin Core serialization, GBIF publishing, data import	SyncJob, ExternalReference, DarwinCoreArchive, ImportBatch

Communication Patterns

- **Frontend to API:** HTTPS REST calls. The Next.js frontend calls DRF endpoints exclusively; no direct database access. Server-side rendering fetches public species data at build time for SEO.
- **API to External Services:** Celery tasks handle all external API calls (IUCN, FishBase, GBIF) asynchronously. Results are cached in Redis and persisted to PostgreSQL. This decouples page response times from external service availability.
- **Admin to Core:** Django Admin connects directly to the ORM for data management. Custom admin actions handle bulk operations (import CSV of holdings, bulk status update).
- **Event-Driven Updates:** When a species assessment or population record changes, Django signals trigger downstream updates (recalculate prioritization scores, flag GBIF re-publish needed, update cached species profile).

Data Model

Key Entities and Relationships

```
Species (1)----(N) ConservationAssessment
   |
   +---(N) ExSituPopulation ---(N) HoldingRecord ---(1) Institution
   |           |
   |           +---(N) BreedingEvent
   |           +---(N) Transfer (from/to Institution)
```

```
|           +---(N) Census
|
+---(N) OccurrenceRecord ---(1) Survey ---(1) FieldProgram
|
+---(N) Distribution (PostGIS geometry)
|
+---(N) BreedingRecommendation
|
+---(N) PrioritizationScore
|
+---(N) CommonName (language: en, fr, mg)
|
+---(1) Taxon (family, genus, species; hierarchical)
```

Entity Definitions

Species -- Central entity. Represents a described or recognized undescribed endemic freshwater fish species. Approximately 79 endemic + 94 non-endemic = 173 total in Malagasy freshwaters.

- Fields: scientific_name, authority, year_described, family, genus, endemic_status (enum: endemic, native, introduced), iucn_status, population_trend, cares_listed (boolean), shoal_priority (boolean), description, ecology_notes, distribution_narrative, morphology, max_length_cm, habitat_type, images, external_ids (iucn_taxon_id, fishbase_id, gbif_taxon_key)
- Sensitivity: Public at Tier 1+

ConservationAssessment -- A point-in-time IUCN Red List assessment or recommended status revision.

- Fields: species (FK), category (EX/CR/EN/VU/NT/LC/DD/NE), criteria, assessor, assessment_date, source (iucn_official, recommended_revision), notes
- Sensitivity: Public at Tier 1+

Taxon -- Taxonomic hierarchy using a nested set or MPTT model.

- Fields: rank (family/genus/species/subspecies), name, parent (FK self), common_family_name
- Represents: Bedotiidae, Cichlidae, Aplocheilidae, Anchariidae, Typhleotridae, Pantanodontidae, etc.

Institution -- A zoo, aquarium, research organization, breeding program, or hobbyist network.

- Fields: name, institution_type (enum: zoo, aquarium, research_org, hobbyist_program, ngo, government), country, city, zims_member (boolean), species360_id, eaza_member, aza_member, website, contact_email
- Sensitivity: Public at Tier 1+ (name, type, country); contact details at Tier 3+

ExSituPopulation -- A population of a species held at a specific institution or breeding program. The key tracking unit for conservation coordination.

- Fields: species (FK), institution (FK), count_total, count_male, count_female, count_unsexed, date_established, founding_source, genetic_diversity_notes, studbook_managed (boolean), breeding_status (enum: breeding, non-breeding, unknown), last_census_date
- Sensitivity: Aggregated counts public at Tier 1+; per-institution detail at Tier 3+; genetics at Tier 4+

HoldingRecord -- A time-series census record for a population, enabling trend tracking.

- Fields: population (FK), date, count_total, count_male, count_female, count_unsexed, notes, reporter (FK User)
- Sensitivity: Tier 3+

BreedingEvent -- Records successful or attempted reproduction.

- Fields: population (FK), date, offspring_count, offspring_survived_30d, method (natural, induced), notes
- Sensitivity: Tier 3+

Transfer -- Movement of individuals between institutions.

- Fields: species (FK), from_institution (FK), to_institution (FK), date, count, purpose (enum: breeding, display, repatriation, rescue), notes, approved_by (FK User)
- Sensitivity: Tier 3+

FieldProgram -- An in-situ conservation project (e.g., Fish Net Madagascar, Durrell Nosivolo, Lake Tseny).

- Fields: name, description, lead_institution (FK), region, start_date, status (active/completed/planned), focal_species (M2M Species), partner_institutions (M2M Institution), funding_sources
- Sensitivity: Public at Tier 1+ (summary); detailed reports at Tier 2+

Survey -- A specific field survey event within a program.

- Fields: field_program (FK), date, site_name, site_location (PostGIS Point), methodology (enum: traditional_fishing, edna, visual, seine_net, electrofishing), surveyor, notes
- Sensitivity: Site location generalized at Tier 1-2; exact coordinates at Tier 3+

OccurrenceRecord -- Darwin Core-compliant species observation. The primary unit for GBIF publishing.

- Fields: species (FK), survey (FK), darwin_core_fields (JSONB for full DwC compliance), location (PostGIS Point), location_generalized (PostGIS Point, for public display), date, individual_count, basis_of_record, collection_code, catalog_number, identification_qualifier
- Sensitivity: Generalized location at Tier 1-2; exact location at Tier 3+

BreedingRecommendation -- A conservation coordinator's recommendation for breeding action on a species.

- Fields: species (FK), recommendation_type (enum: establish_population, increase_population, genetic_rescue, managed_breeding, reintroduction), priority (enum: critical, high, medium, low), target_institutions (M2M Institution), recommended_by (FK User), date, rationale, status (open, in_progress, completed)
- Sensitivity: Tier 3+

PrioritizationScore -- Composite conservation priority score integrating multiple criteria.

- Fields: species (FK), iucn_score, ex_situ_gap_score, genetic_distinctiveness, shoal_priority (boolean), cares_listed (boolean), range_restriction_score, trend_score, composite_score, scoring_methodology_version, date_calculated
- Sensitivity: Public at Tier 1+ (composite score); component scores at Tier 2+

User -- Platform user with tiered access.

- Fields: email, name, institution (FK, nullable), access_tier (1-5), is_active, expertise_areas, orcid_id
- Five tiers as specified in CLAUDE.md

AuditLog -- Immutable record of data modifications for accountability.

- Fields: user (FK), action (create/update/delete), model_name, object_id, timestamp, changes (JSONB), ip_address

Storage Strategy

- **PostgreSQL + PostGIS** for all structured data. PostGIS geometries for species distributions (polygons), survey sites (points), and protected areas/KBAs (polygons).
- **JSONB columns** on OccurrenceRecord for full Darwin Core compliance without requiring schema changes for every DwC term. The core DwC terms used most frequently (scientificName, decimalLatitude, decimalLongitude, eventDate, basisOfRecord) are also stored as indexed columns for query performance.
- **S3-compatible object storage** for images, PDFs (field reports, published papers), and generated Darwin Core Archive ZIP files.
- **Redis** for session data, cache, and Celery task queue.

Coordinate Generalization

Following GBIF sensitive species best practices, occurrence locations for threatened species are generalized based on access tier:

Tier	Location Precision
Public (Tier 1)	Country + broad region only (e.g., "Sofia Region, Madagascar")
Researcher (Tier 2)	Generalized to 0.1 degree (~11 km)
Coordinator (Tier 3+)	Exact coordinates

Generalization is applied at the API serializer level -- the database stores exact coordinates, and the DRF serializer returns the appropriate precision based on the authenticated user's tier.

Integration Points

Read Integrations (data pulled into platform)

External System	Method	Data	Frequency	Notes
IUCN Red List API	REST API (v4, token-based)	Species assessments, categories, criteria, habitats, threats, conservation actions	Weekly sync via Celery	Free for non-commercial use. Rate limited; responses cached 7 days. ~79 endemic species to track.
FishBase	rfishbase API / REST	Morphology, ecology, distribution, common names, trophic level, habitat preferences	Weekly sync via Celery	Coverage for Malagasy endemics is thin; platform supplements with local data. Cached responses.
GBIF Species API	REST API	Taxonomic backbone matching,	On-demand + weekly sync	Used to reconcile taxonomy and link to GBIF taxon keys.

		occurrence counts, dataset references		
ZIMS / Species360	Institutional data-sharing agreement (CSV/manual)	Captive population snapshots, institutional holdings	Quarterly manual import	No public API. Requires membership or data-sharing agreements with participating institutions. CSV import pipeline in the integration app.
Zootierliste	Web scraping or manual import	Additional captive holdings not in ZIMS (private zoos, rescue stations)	Semi-annual manual import	Augments ZIMS data per Leiss et al. methodology.
CARES Priority Lists	Manual import (HTML scrape or CSV)	Species priority status, regional coordinator info	Annual update	Static HTML lists on WordPress; no API. Import tool parses their published lists.

Write Integrations (data published from platform)

External System	Method	Data	Frequency	Notes
GBIF	Darwin Core Archive + GBIF IPT	Occurrence records	On-demand publish via Celery task	Generated DwC-A files registered with GBIF Integrated Publishing Toolkit. Sensitive coordinates generalized per GBIF best practices before publishing.
SHOAL Database	Manual export or future API	Conservation project data	As needed	SHOAL's data officer (Georgie Bull) is building a freshwater fish conservation project database. Platform provides structured export for ingestion.

Planned Future Integrations

System	Status	Notes
Citizen Conservation "Wild at Home"	Watch	Currently in password-protected beta. When API becomes available, integrate for private breeder inventory sync.
BiOS Platform	Watch	Emerging 2026 biodiversity platform with PostgreSQL-PostGIS backend and REST API. Potential interoperability partner.
eBird/iNaturalist model	Aspirational	Community-contributed occurrence records with review workflow. Low priority until user base grows.

Security & Access Model

Authentication

- **Primary:** Django's built-in authentication with session-based auth for the web frontend and token-based auth (DRF TokenAuthentication or JWT via SimpleJWT) for API access.
- **Registration:** Open self-registration for Tier 1 (public, no account needed) and Tier 2 (registered researcher, email verification required). Tier 3+ requires approval by a Tier 5 administrator.
- **Institutional SSO:** Not required at launch. If adoption grows among EAZA/AZA member institutions, SAML or OIDC integration could be added.
- **Password Policy:** Minimum 12 characters, bcrypt hashing (Django default with PASSWORD_HASHERS), rate-limited login attempts.

Authorization: Five-Tier Access Model

The access model follows Symbiota's five-tier pattern, adapted for conservation coordination as specified in the CLAUDE.md:

Tier	Role	Can View	Can Edit	Typical User
1	Public (anonymous)	Species profiles, conservation status, general distribution maps, aggregated ex-situ counts, field program summaries, prioritization composite scores	Nothing	General public, educators, journalists, funders
2	Registered Researcher	All Tier 1 + occurrence datasets, published survey data, field program detailed reports, component prioritization scores, generalized (0.1-degree) locations	Own occurrence record submissions (pending review)	University researchers, graduate students, conservation consultants
3	Conservation Coordinator	All Tier 2 + exact locations, per-institution population detail, breeding recommendations, transfer records, census data, action items	Population records for affiliated institutions, field survey data, breeding recommendations, occurrence records (direct publish)	TAG coordinators, EEP managers, CARES regional coordinators, Fish Net Madagascar team, SHOAL staff

4	Program Manager	All Tier 3 + genetic diversity data, pedigree information, studbook-level records, institutional inventory detail	All Tier 3 + genetic data, studbook records, breeding program configuration, bulk data import	Studbook keepers, Species360 institutional representatives, Citizen Conservation program leads
5	Administrator	Full system access	Full system access + user management, tier assignments, system configuration, data deletion, audit log access	Platform maintainers, project leads

Implementation

- Access tier is stored on the User model as an integer field (1-5).
- DRF serializers use `SerializerMethodField` to conditionally include/exclude fields based on `request.user.access_tier`.
- Django model managers provide `.for_tier(tier)` querysets that filter sensitive records.
- A custom DRF permission class `TierPermission(min_tier=N)` gates endpoint access.
- Institution-scoping: Tier 3-4 users can only edit records associated with their affiliated institution(s). This is enforced at the queryset level, not just the view level.

Data Sensitivity

Data Category	Storage	Access	Notes
Species profiles, IUCN status	PostgreSQL	Public	Core public-facing content
Occurrence coordinates (threatened species)	PostGIS (exact)	Exact at Tier 3+; generalized at Tier 2; region-only at Tier 1	Following GBIF coordinate generalization protocols
Captive population counts (aggregate)	PostgreSQL	Public (Tier 1)	Total count by species, no institution detail
Captive population detail (per-institution)	PostgreSQL	Tier 3+	Institution names, individual counts, breeding status
Genetic diversity data, pedigree	PostgreSQL	Tier 4+	Highly sensitive for breeding management
Breeding recommendations, transfer plans	PostgreSQL	Tier 3+	Active coordination data
User PII (email, name)	PostgreSQL	Tier 5 for other users' PII; own profile at any tier	Minimized; ORCID used for researcher linking

Audit logs	PostgreSQL (append-only)	Tier 5	Immutable; retained indefinitely

Encryption

- **In transit:** TLS 1.3 enforced on all connections (HSTS headers, secure cookies).
- **At rest:** Database-level encryption via PostgreSQL TDE or cloud provider encryption (e.g., AWS RDS encryption). S3 bucket encryption for file storage.
- **Secrets:** Environment variables for API keys (IUCN, GBIF tokens), database credentials, Django SECRET_KEY. Never committed to version control. Managed via environment files (.env) excluded by .gitignore.

Deployment & Infrastructure

Target Environment

The platform should be deployable in two modes to accommodate the conservation community's resource constraints:

1. **Cloud-hosted (primary):** A low-cost cloud deployment suitable for a small-user-base conservation platform. Expected traffic is low (hundreds of users, not thousands), so infrastructure costs should be minimal.
2. **Self-hosted:** Docker Compose-based deployment for organizations that need to run their own instance (e.g., a Malagasy NGO operating in-country per the January 2026 workshop discussion about establishing a local NGO).

Cloud Infrastructure (Recommended)

Component	Service	Cost Estimate
Application server	Fly.io or Railway (Django + Celery)	$5-20/month
Database	Fly.io Postgres or Supabase (PostgreSQL + PostGIS)	$0-15/month
Redis	Fly.io Redis or Upstash	$0-5/month
File storage	Backblaze B2 (S3-compatible)	~$0 at this scale
Domain + DNS	Cloudflare	Free
CDN	Cloudflare	Free
SSL	Let's Encrypt via Cloudflare	Free

Estimated total: $5-40/month -- critical for a conservation platform dependent on grant funding and institutional contributions.

Alternative: If institutional hosting is available (e.g., through a university partner like University of Antananarivo or a zoo IT department), a single Linux server with Docker Compose is sufficient for the expected load.

Containerization

```
docker-compose.yml
  services:
    web:        Django application (Gunicorn)
    api:        Same Django image, serving DRF endpoints
    frontend:   Next.js (can also be statically exported)
    worker:     Celery worker (same Django image)
    beat:       Celery beat scheduler
    db:         PostgreSQL + PostGIS
    redis:      Redis
    minio:      MinIO (dev only; replaced by cloud S3 in production)
```

Single Docker image for Django serves web, API, and worker roles -- differentiated by entrypoint command. This minimizes build complexity.

CI/CD

Stage	Tool	Trigger
Lint + Type Check	Ruff (Python), ESLint + tsc (TypeScript)	Every push
Unit Tests	pytest (Django), Vitest (Next.js)	Every push
Integration Tests	pytest with PostgreSQL test database	Every PR
Security Scan	pip-audit, npm audit, Trivy (container)	Every PR
Build	Docker multi-stage build	On merge to main
Deploy	Fly.io CLI or Railway CLI	On merge to main (staging); manual promote to production
Darwin Core Validation	Custom pytest fixtures validating DwC-A output against GBIF validator	Every PR touching integration app

Environments

Environment	Purpose	Database
Local dev	Developer machines, Docker Compose	Local PostgreSQL + seed data
Staging	Pre-production testing, demo to stakeholders	Cloud PostgreSQL, seeded with anonymized production data
Production	Live platform	Cloud PostgreSQL with automated backups

Backup Strategy

- **Database:** Automated daily backups with 30-day retention. Point-in-time recovery enabled.
- **File storage:** S3 versioning enabled; cross-region replication if budget allows.
- **Configuration:** All infrastructure defined as code (Docker Compose + deployment scripts in repository).

Risks & Open Questions

Technical Risks

Risk	Severity	Mitigation
ZIMS has no public API. Captive population data -- the core of ex-situ tracking -- depends on manual imports from institutional data-sharing agreements.	High	Build robust CSV/Excel import pipeline with validation. Design the data model so ZIMS data is clearly sourced and timestamped. Engage Species360 early about potential data-sharing partnership.
IUCN Red List API rate limits and coverage gaps. Roughly a quarter of species are Data Deficient; many assessments are outdated.	Medium	Cache aggressively (7-day TTL). Store local assessment overrides for recommended revisions (per Ziegler et al. 2020). Clearly label data source and freshness in the UI.
Citizen Conservation "Wild at Home" overlap. CC is building a similar (broader) platform. If Wild at Home launches with comprehensive features, this platform could be redundant for ex-situ coordination.	Medium	Design for interoperability from day one. If Wild at Home exposes an API, integrate rather than compete. Focus this platform's differentiation on Madagascar-specific species profiles and in-situ/ex-situ integration that Wild at Home will not provide.
Small contributor pool. The user community is small (~50-100 active contributors globally). Platform adoption depends on key individuals (Fusari, Ziegler, Encke, Ravelomanana).	Medium	Ship an MVP quickly that delivers immediate value (species profiles + basic population tracking). Use Django admin for early data entry to reduce frontend development time. Seed the database with Leiss et al. 2022 baseline data.
Darwin Core extension complexity. Standard DwC does not cover breeding coordination data (genetics, pedigree, husbandry). Custom extensions require careful design to remain interoperable.	Medium	Use DwC for occurrence/species data where it fits. Define a separate, documented schema for breeding coordination data rather than forcing it into DwC. Maintain DwC compliance for the GBIF-publishing subset of data.
Madagascar connectivity. In-country partners face intermittent internet. Field data entry may need offline support.	Low (initial)	The platform is primarily used by the international coordination community (Europe, North America). In-country field data is collected during expeditions and entered after return. Offline support (PWA or mobile app) is a future enhancement, not MVP.

Open Questions Requiring Human Input

1. **Governance model (initial phase resolved).** The project lead (Aleksei Saunders) will own and host the platform at launch, positioning it as an MVP proposal to present to stakeholders. Long-term handover — to the informal Madagascar Freshwater Fish Conservation Group, SHOAL, a proposed Malagasy NGO, or an existing institution (e.g., Cologne Zoo, ZSL) — remains open and will be negotiated with stakeholders after the MVP is demonstrated. **Architecture implication:** hosting

setup should be straightforward to transfer (avoid lock-in to personal accounts; prefer org-level credentials from the start).

2. **SHOAL database coordination.** SHOAL has hired a data officer (Georgie Bull) who is building a freshwater fish conservation project database. Should this platform integrate with SHOAL's database, feed into it, or maintain independence? The January 2026 workshop noted that SHOAL will send a link for adding project information. **Needs clarification on SHOAL's data architecture and API plans.**

3. **Relationship with the June 2026 ECA Workshop.** The CPSG Ex-situ Conservation Assessment Workshop (ABQ BioPark, June 1-5, 2026) will assess ~360 globally imperiled freshwater fishes. Malagasy species will be included. The platform could serve as the data infrastructure for workshop outcomes and ongoing ECA tracking. **Needs engagement with NMBPS/CPSG workshop organizers to align data models.**

4. **Licensing.** The platform is described as open-source, but the specific license needs selection. AGPL-3.0 ensures derivative works remain open but may deter institutional adoption. MIT/BSD is more permissive but allows proprietary forks. Apache-2.0 is a common middle ground. **Needs decision from project lead.**

5. **Data entry responsibility.** Who enters and maintains species profile data, population records, and field program updates? Options range from a dedicated data manager (requires funding) to distributed community entry (requires review workflows) to automated imports (limited by API availability). The January 2026 workshop raised this explicitly: "who will implement it and how?" **Needs staffing/resource plan.**

6. **Prioritization tool integration.** The January 2026 workshop discussed a scoring system for species prioritization, including SHOAL 1,000 Fishes as a criterion. The platform should host this scoring tool, but the exact criteria and weights need to be defined by the conservation community, not the development team. **Needs species prioritization criteria from the working group.**

7. **KBA/Protected Area spatial overlay.** Thomas Ziegler proposed overlaying species distribution with protected areas and Key Biodiversity Areas. PostGIS supports this natively, but the source spatial data (Madagascar protected area boundaries, KBA polygons) needs to be identified and licensed. Protected Planet (WDPA) provides some of this data. **Needs confirmation of spatial data sources and licensing.**

8. **Multilingual support scope.** Users are primarily English-speaking (international researchers, zoo staff), French-speaking (Malagasy researchers, French institutions), and Malagasy-speaking (in-country partners). Species common names exist in all three languages. Full UI internationalization adds development cost. **Needs decision on MVP language (English) vs. multilingual from the start.**